

Transcript

Module 8 Week 9: Working with React – Part 1

Video 1: Module Overview

Welcome to the Working with React module. This module will teach you one of the most popular libraries used for building single page applications, React. After taking this course, you should gain a solid understanding of what React is, why it's so powerful, and how to use it to effectively build modern web applications.

We will start with an introduction to React. Before you can build with React, it's essential to know what React is. We will explore the core principles behind React and what makes it stand out as a library. Once you understand the basics, we will move on to setting up a React environment. You will learn about components based design. Next, we will cover one of React's most important feature, props. Props allows you to pass data from a component to another, making your application more dynamic. To understand how files in React are organised, we'll introduce the ECMAScript 6 or ES6 module system and teach you how to import and export modules. Even though this course focuses on React, we often use various ES6 constructs in development. In this segment, we will look at key ES6 features that are widely used in React, such as arrow functions and destructuring.

At the end of week 1, we will cover a hands on exercise where you will learn how to build a simple profile viewer app using the concept covered in week 1. We will then explore event handling in React. You will learn how to capture and respond to user actions such as click or form submission. To make an app dynamic, React relies on something called state. In this topic, we will discuss state management using React Hooks. You will learn how to define and manage state in your component. When designing components, React offers controlled and uncontrolled components for managing user input. You will learn about the differences between the two and how to use them effectively to ensure predictable behaviour in your forms and interactive elements.

A React app often involves communicating with a server. In this topic, we will learn how to make requests using the Fetch function and how to handle the response to dynamically update your React apps with external data. This web communication is often achieved using the useEffect hook. You will learn how to handle side effects like data fetching in your component using the useEffect hook. We will also introduce the useRef hook, which allows you to interact with the DOM element directly and cover the essential rules for using hooks. React Router allows you to create multi page functionality within your single page app. We'll explore how to set up, browse, and navigate between different views, maintaining a smooth user experience.

Finally, we will look at Create React App, a tool that simplifies setting up a new React project. You will see how to quickly create a React app and how this tool is valuable for developers. By the end of this module, you will have a solid understanding of the React fundamentals. I'm excited to see what you will be creating as you continue in this journey.

Video 2: Introduction to React

Welcome to the lecture on Introduction to React. Here are the learning objectives. By the end of this lecture, you should have a clear understanding of what React is, the benefits it offers, how to set up a webpage to work with React, what JSX is, what transpiling is, and how to create components in React.

React is a JavaScript library used for creating user interfaces. It was developed by Facebook and was first released in 2013. One of the key features of React is its components based architecture, which allows you to create encapsulated components that manage their own state. In large scale applications, front end development can be quite challenging. We need a systematic way to manage this complexity. React helps to manage the DOM, automatically providing a declarative way of development. This means you can define how you expect the page to look like based on the data. And React will take care of the rendering and re-rendering when the data changes.

React has several key features that stand out. It is components based, which promotes reusability and modularity. It uses a virtual DOM to improve performance by minimising direct manipulation of the actual DOM. React uses JSX, a syntax that allows developers to write HTML like codes in JavaScript. Lastly, React promotes a unidirectional data flow, making data management more predictable and easier to debug. React offers several benefits. First, the reusability of components makes development more efficient. Second, the virtual DOM improves performance by reducing the number of operations on the actual DOM. JSX makes it easier to write and understand the structure of your UI. Finally, React has a large and active community, which means a wealth of resources and support is available.

Now, let's get started with React. The first step is to add React to your project. We will need to include the CDN link from React and ReactDOM to the head section of our HTML document. Next, add the container tag to the body section, typically a div with an ID of root. Finally, add a script section in the body where we will write our React code. Here is a simple example to render a Hello World message. We normally define a element where React will do the rendering, the `render` method takes a single react element for rendering. Here is an example of rendering an unordered list with tool items.

While we use `react.createElement` to create our elements, it's more convenient to use JSX instead of writing the following. We would prefer to write the code as the following instead. JSX is a syntax extension that allows us to write HTML like code in JavaScript. However, JSX is not valid JavaScript, so it needs to be converted into proper JavaScript before the browser can understand the codes. Transpiling is the process to convert JSX into valid JavaScript, which consists of the `react.createElement` construct internally. To use JSX in our project, we will need to add Babel transpiler. We need to first include the Babel script in the head section and specify `text Babel type` attribute in the script tag. This will trigger the Babel transpiler to transform the JSX into valid JavaScript, which the browser can understand.

JSX is very similar to HTML, but there are some differences. For example, instead of specifying the `class` attribute, you need to use the `className` attribute. Instead of the `for` attribute, you need to use the `htmlFor` attribute. Additionally, when specifying the `style` attribute, the value should be a JavaScript object with camelCased property instead of a string.

Now let's combine everything that we have learned. Our HTML file should include React, ReactDOM, and Babel script in the head section. The body section should contain a div with id equals to root and a script tag with a type attribute having the type Babel value. Pause the video now or rewind this video to make sure that you can see the following output. In the next screen, we will show you how the complete HTML should look like. This is how the HTML should look like. In this section, we will discuss how components based design in React look like. In React, components are the basic building blocks for creating interfaces. In a React application, the entire UI is often made up with components, and the components can consist of other components. React Components extends the HTML vocabulary, allowing you to write HTML within JavaScript codes.

There are two main ways to define components, using ES6 Class Components or Function Components. For this module, we'll focus on creating Function Components. While Class Components used to be important, they are not needed anymore and might not be supported in the future. Here is an example of how you can define a component using the class component syntax. After we have defined the components, we can use the component by defining the menuComponent tag similar to a normal HTML tag.

Function component syntax is another approach of defining components. This syntax is easier and more straight forward. Like the name suggests, we define our component using a function where the name of the function is the name of the component. In here, we define the menuComponent as the menuComponent function. This function should return the JSX to be rendered. You can perform any logic before returning the JSX and you can even define nested functions.

Let's build a React component together. Try writing the codes that you see in the previous screen and then use the component. Ensure that the menuComponent is properly rendered as shown in the screen. Again, pause this video and try this out. We will review the JavaScript that is needed to achieve this in the next screen. This is how the HTML should look like. In summary, we have covered what React is, the benefit it offers, how to set up a web page to work with React, what JSX is, what transpiling is, and how to create components in React.

Video 3: Working with React Props

Welcome to a lecture on working with React props. Here are the learning objectives. By the end of this lecture, you should understand what props are, the purpose of using props, and how to evaluate JavaScript expressions in React using the curly braces operator.

One of the main benefits of using components is their reusability. To make components truly reusable, we need to find a way to pass data to them so that the components can behave differently with different data values. This is where props comes into play. Props, short for properties, allows us to pass values to components. Consider the following code snippet. Here we are passing the first and last props to the GreetingComponent. The component receiving the props will specify a function parameter usually named props. Within the function, we can obtain the props that will pass into the component by assessing the props parameter. This parameter is an object containing the property value pairs. In this example, props.first and props.last are used to assess the first and last props, respectively.

In React, we often need to evaluate JavaScript expressions within our JSX. To do this, we enclose the expression within curly braces. The following contains some examples how we can go about doing this. Here, the value of n, str, and percent are evaluated and rendered within the JSX. When using the components, note that when passing a non string value as props, we also need to use the curly braces operator. For instance, if you pass numeric values or objects, they must be enclosed within curly braces. In this example, field2, field3, and field4 are passed as props using the curly braces operator.

When working with array, we can render them using the map function. Here is an example. In this component, we map over the data array and render each item as a list element. Each element should have a unique key. If we supply the second argument for the map function, we will be able to obtain the index of the item in the data array. Pause this video now and try out the previous example to make sure that you are able to render a list of items supplied to a component as props. It is important to note that if we do not include the key props when rendering a collection, React will display a warning as shown here. Always ensure to provide a unique key for each element.

This allows React to optimise the rendering process and avoid unnecessary re-rendering. Other than rendering an array of items, sometimes we might want to render multiple elements. Remember that JSX expression must always have a single root element. If we need to return multiple elements, you can use the React fragment. React fragment can be thought of as a tag with no tag name. By using React fragments, ensures that our JSX has a single root element while allowing you to return multiple elements.

Sometimes instead of using attributes, we might want to pass large chunks of content into a component. This is when `props.children` come in handy. This is an example of how we can do this. In here, the `RoundBorderComponent` receives the props within the starting and closing tag rather than through attributes. In this component, we want to display the supplied HTML element with a round border. Recall that JSX is very similar to HTML, but there are some key differences. For instance, the `style` attribute in JSX should be a JavaScript object with camelcased properties.

In summary, we have covered what props are, the purpose of using props, and how to evaluate JavaScript in React using the curly braces operator. Props are essential for passing data to components, making them reusable and dynamic. Understanding how to work with props is crucial for building efficient and maintainable React applications.

Video 4: ES6 Module System and Organising React Applications

Welcome to lecture on module system and organising React applications. Here are the learning objectives. By the end of this lecture, you should have a good understanding of how to work with ES6 module system, how to organise React application, and how to create React application without setup. This skill will enable you to develop more maintainable and scalable React application. This video will involve several hands on segments where you will get to learn by doing. Please consider pausing the video at any point in time to try out the code examples to reinforce what you have learned.

In React development, it is a common practice to organise each component in its own file. For example, if we have a `RoundBorderComponent`, we will store it in a file named `RoundBorderComponent.js`. Each file can be considered as a module. ES6 provides a built in support for module systems using the `export` and `import` syntax. which allows us to share and reuse code efficiently. When creating React components, we typically export them from their respective file using the `export default` syntax. Here is an example of how we might define and export a `GreetingComponent`. By specifying `default` in the `export` syntax, would mean that when this module is imported later on, it will be the default module that will be imported even though we do not specify the name of the module.

JavaScript also allows us to specify a named export, in which case the name of the module needs to be specified during import. To keep things simple, most of the time we will do `export default`. To use a component from another file, we will import it by first providing the relative path to the module.

For instance, to use our `GreetingComponent`, we will do the following. In this code segment, when we are importing the `GreetingComponent`, We also specify a name which is used to then refer to this imported module. Note that this name can differ from the function name that is defined in the `GreetingComponent.js` file. However, to avoid confusion, the best practice is to specify a name that matches the file name. We can then render this component within our JSX using the `GreetingComponent` tag, which will output a text based on what is specified in the `name` prop.

Now we will discuss how to get started with developing with React without setup. One of the easiest ways to start coding in React without setup is to use CodeSandbox. CodeSandbox is an online editor tailored for developing web applications. Unlike HTML, CSS, and JavaScript, React does require some setting up for it to work properly. CodeSandbox takes care of this setting up and allows you to get started with developing React applications without having to go through the software download and installation process. You can access CodeSandbox by visiting the following URL. CodeSandbox utilises CreateReactApp under the hood.

We will delve into Create React app later on. But for now, when using CodeSandbox, our starting point is under the source or src folder and the index.js file. The best way to learn coding is to learn by doing. We will encourage you to do some hands on by pausing this video and trying out the codes on CodeSandbox. Most of the time, we will be working with the source folder.

First, create a new file and name it GreetingComponent.js within the source folder. Next, implement the GreetingComponent.js file with the following code as seen in the earlier videos. These GreetingComponents take in an argument capturing the props that is supplied to the component. We then display the name that is supplied to this component by using the name prop. The next step is to use this component. Before that, we need to understand how React Apps are usually organised.

While index.js is the starting point of the React app, to better support components based design in React, the initial setup of React apps usually consists of another app component, which is imported and used in index.js. You can think of it as index.js is a starting point and app is the main component that powers the entire React app. So we will modify app.js to use GreetingComponent. Similar to how things are done in index.js, we will first need to import GreetingComponent. Next, we will then write the JSX that contains the GreetingComponent tag and supply a name prop.

If everything works correctly, your preview panel on the right should display a message showing Hello John, as shown here. Pause this video to ensure that you can get this simple React app working on your CodeSandbox account. If it doesn't work, please consider rewinding this video to ensure that you have created a file in the appropriate folder and have implemented the file properly.

In conclusion, we have covered the ES6 module system, how to organise React applications, and how to create React applications without setup. These concepts are fundamental to developing modern React applications.

Video 5: Modern ES6 Constructs

Welcome to the lecture on Modern ES6 Construct. Here are the learning objectives. By the end of this lecture, you should have a good understanding of arrow function, template strings, object literal short-hands, rest and spread operators, and de-structuring. First up, we have arrow functions. Arrow functions provide a new, shorter syntax for function declarations. This syntax is somewhat similar to lambda expressions languages like C# and Python.

For instance, instead of writing a function to add up two numbers like this, we can simplify it by using an arrow function. Notice how the return statement is optional and assumed. Now instead of using a normal function for the map function, try using an arrow function for your map function. Pause this video and use your web browser developer console to try this exercise. We will discuss the solution in the next screen. Here is how you can achieve this using an arrow function. Next, we will move on to template strings. Template strings allows us to use backticks to create multi-line strings and support string interpolation given the following.

Example, we can create a multi-line string by enclosing the string within backticks by using the dollar sign and curly braces operators. We can obtain the value of add identifier. For example, we can replace the dollar sign, curly braces `person.name` and `person.location` with the value of the name and location property of the person object. This is a lot more convenient as we do not need to resort to string concatenation. Note that the string interpolation operators also works with any JavaScript expressions. Specifically, we can put any JavaScript expression within the dollar sign curly braces operators. Here are some more examples of how string interpolation operators can be used. Update the code of the previous exercise to use template string in your map function so that you can avoid string concatenation. Pause this video and try this out. We will discuss the solution in the next screen. Here is the solution using template string.

Next, we will discuss object literal notation. ES6 introduced shorthand notation for object properties. Previously, you might have written the following. Now with ES6, we can simplify it to the following. Basically, if the property of an object is the same as the identifier name, we can specify the name once. Next, let's move on to the rest and spread operator. First, the rest operator allows us to group things up together. For example, in the following function, we define three arguments for the `generateGreetings` function, `name`, `gender`, and `items`. Notice that the last argument contains three periods in front of it. In this example, `name` will match the first parameter, `gender` will match the second parameter and `items` will match the rest of the remaining function parameters. So when this function is called, the `item's` argument will group up the third function parameters onwards into an array containing `iPad` and `iPhone`.

However, if fewer than three parameters are provided, the `item` argument will be an empty array. The spread operator does the opposite. Instead of grouping values, it spreads them out. For instance, with arrays, we can spread out the values of an array. In here, by spreading out the value of `array1` and `array2`, we can create a new array consisting of 0, value of `array1`, value of `array2`, and another string `d`. This is an easy way to perform array concatenation without using any functions.

Similar to the rest operator, the spread operator uses three periods in front of identifier. But due to where this is being applied, this becomes a spread operator instead. The same idea applies for objects where the properties of an object is spread out. In this example, we spread out the properties of `personName` object and the `personContact` object and use the property to create a new object. Essentially, this combines the properties of the two objects into a new object.

Next, let's look at destructuring. ES6's destructuring assignment allows for easy matching of arrays of properties from objects and assigning them to variables. Let's start with an array example. In here, we destructure the first item of an array into `available` call `first`, and the second item of the array into `available` call `second`. Destructuring can also be used for object properties.

Similar to the array case, we can destructure the properties of an object into `available` using the name of the property. This makes things a lot easier. Previously, we would have to write something like this. Assigning `Person.firstName` to `firstName`. But with object destructuring, we can use the curly braces to destructure out the `firstName` property into a variable called `firstName`. Notice that the variable name and the property name matches. While this is not absolutely needed, it is especially helpful to take away the need to have to do additional checking in case we are not able to find a property during the destructuring process. In which case the variable will be assigned to the specified default value. Destructuring can also be used in function parameters, allowing for more concise code.

In summary, we have covered the various commonly used ES6 constructs such as arrow function, template string, object literal shorthand notation, rest and spread operators, and destructuring. This construct will help you write cleaner codes. You will see many of these constructs in the codes that you find online.

Video 6: Hands on Exercise Building a Profile Viewer

Welcome to the video on hands on exercise building a profile viewer. We will be building a profile viewer together in this video. Here are the learning objectives. By the end of this lecture, you should be able to understand how to build a simple React app from scratch. Let's get started. Here's a sneak preview of what we will be building in this video. This is how the profile viewer will look like once we have completed this exercise. Let's break it down step by step.

You're encouraged to work on the exercise while watching this video. Pause this video at any point in time to try out the steps on your machine. First, we'll be using CodeSandbox to create our React app. For simplicity, assess the following URL to create a new React sandbox on CodeSandbox platform.

Next, we will need some data to work with. For this exercise, we will use an array of hard coded user records. This data is taken from the following random-data-api endpoint at the time of this video recording. You can also come up with your own dummy data. The endpoint will generate three random user records, so your data will mostly look different from this screen. Here we have also manually removed away other properties that will not be used for this video, leaving behind eight properties, namely id, first_name, last_name, email, avatar, phone_number, date_of_birth, and employment. However, you can just copy the entire array without modification.

Now copy the people array to app.js that's residing in the source folder. Remove away any unnecessary codes from app.js. Here is a screenshot of how your app.js should look like after this step. Next, create a new file named profile.js in the source folder and add some placeholder codes in this file. Here's an example of how this file might look like.

Now, let's use our profile component in app.js. First, import the profile module and then use the profile module in the body of the function. It should look something like this. The profile component should take in one user record, through prop, and display the user profile. The next step is to modify app.js to use the map function on the people array. Don't forget to add the key prop. Your code should look something similar to this. As we have previously defined some placeholder codes to you should see the placeholder text in each of the three user profiles.

Now, let's focus on the styling design of the profile component. We will define four styles as normal JavaScript functions, namely container style, image container style, image style, and description container style. The following screenshot shows the location by which the four styles will be applied. Head over to profile.js and define the following JavaScript object for the container style. It should be a normal JavaScript object that is defined within profile.js. Next, define the imageContainerStyle object in profile.js as shown here. Then, define the imageStyle object in profile.js. Finally, define descriptionContainerStyle object in profile.js. After defining all the styles, profile.js should look like this.

The next step is to pass the property of a person to the profile component. Using the destructuring construct, we can then destructure the individual fields from the properties. After defining the styles and destructuring the fields, we are ready to structure the UI of the profile component. This screen shows how we can lay out the HTML element to apply the 4 Style defined.

This is how the JSX should look like. If everything works out, your app should look like this. Pause this video to ensure that you are able to see something like this before proceeding further. If you are unable to get something like this, try rewinding this video and review the steps earlier.

Now let's add the avatar to the profile component. By setting the source attribute of the image tag, we can define the image which acts as an avatar image. Note that the avatar field has been destructured from the props earlier. Now it's your turn. Similar to how we have added avatar, modify the JSX to include the remaining fields. For the employment prop, you will need to further destructure out the title and key_skill fields.

In this hands on exercise, we have covered how to build a simple React app from scratch. We have walked through creating a new React sandbox, working with dummy data, building and styling a profile component. I hope that this exercise is helpful, and it gave you a better understanding of how to work with React components and props. Thank you.

THE END